

Profiling de técnicas de otimização de código

Planejamento fatorial 4×2 com `perf` sobre multiplicação de matrizes

Disciplina	SSC0951 - Desenvolvimento de Código Otimizado
Docente	Hermes Senger
Alunos	Samuel Rubens Souza Oliveira (NUSP 11912533) José Guilherme Santos Riberto (NUSP 15647503)
Atividade	Relativa à aula 3 (Profiling)

1 Enunciado da atividade

O objetivo é o estudo das técnicas de otimização de código apresentadas na aula 3, Loop Interchange e Loop Unrolling, utilizando o perfilador `perf` e técnicas de análise de resultados. A configuração dos experimentos é fixada pelo enunciado.

Fator 1, técnica de otimização de código, com quatro níveis: sem otimização, Loop Interchange, Loop Unrolling e Loop Tiling.

Fator 2, alocação da matriz na memória, com dois níveis: estática e dinâmica.

Métricas a coletar em cada experimento: tempo de resposta, `L1-dcache-loads`, `L1-dcache-load-misses`, `branch-instructions` e `branch-misses`.

O cruzamento dos dois fatores define oito experimentos, numerados de 1 a 8 conforme a Tabela 1. Cada experimento deve ser executado pelo menos dez vezes, e deve-se calcular a média e o intervalo de confiança a 95% das métricas coletadas.

As duas análises pedidas

O enunciado encerra especificando duas análises, que são as perguntas que este relatório responde de forma objetiva nas Seções 3 e 4.

1. **Comparação do tempo de resposta nos 8 experimentos.**
2. **Influência da variação dos fatores nas variáveis de resposta de cache**, considerando os experimentos 1, 2 e outros dois com a mesma técnica, à escolha entre [3 e 4], [5 e 6] ou [7 e 8], **e influência da variação dos fatores nas variáveis de resposta de branch**, considerando os experimentos 1, 2 e outros dois com a mesma técnica, também à escolha.

Para a análise de cache foram escolhidos os experimentos **1, 2, 3 e 4**, ou seja Loop Interchange, porque é a técnica que atua diretamente sobre o padrão de acesso à memória. Para a análise de branch foram escolhidos os experimentos **1, 2, 5 e 6**, ou seja Loop Unrolling, porque é a técnica que atua diretamente sobre o controle de laço e portanto sobre a contagem de desvios.

2 Descrição da atividade realizada

2.1 Programa medido

O programa é a multiplicação de matrizes quadradas densas $C = A \cdot B$ com $N = 1.000$ e elementos do tipo `int`. As três matrizes ocupam 11,44 MiB no total, valor muito acima da cache L3 de 12 MiB desta máquina, de modo que o padrão de acesso à memória é o fator dominante de desempenho e as técnicas sob estudo têm efeito mensurável.

Os oito binários correspondem exatamente às oito células do planejamento.

Tabela 1: Mapeamento dos oito experimentos do enunciado para os binários medidos.

Exp.	Técnica	Alocação	Kernel executado
1	sem otimização	estática	<code>multiplyMatrixStatic</code>
2	sem otimização	dinâmica	<code>multiplyMatrix</code>
3	Loop Interchange	estática	<code>loopInterchangeMultStatic</code>
4	Loop Interchange	dinâmica	<code>loopInterchangeMult</code>
5	Loop Unrolling	estática	<code>loopUnrollingMultStatic</code>
6	Loop Unrolling	dinâmica	<code>loopUnrollingMult</code>
7	Loop Tiling	estática	<code>loopTilingMultStatic</code>
8	Loop Tiling	dinâmica	<code>loopTilingMult</code>

2.2 Os quatro níveis do Fator 1

Cada nível aplica **uma única** transformação sobre o mesmo laço base. Essa disciplina é necessária para que o efeito principal da técnica seja atribuível: se um nível carregasse duas transformações ao mesmo tempo, o efeito e a interação ficariam confundidos.

- **Sem otimização:** ordem i - j - k . O laço interno percorre uma linha de A e uma *coluna* de B , com passo de 4.000 bytes.
- **Loop Interchange:** única mudança é trocar j e k , resultando em i - k - j . O laço interno passa a percorrer uma linha de B e uma linha de C , ambas sequenciais, e `a[i][k]` fica invariante.
- **Loop Unrolling:** mantém a ordem i - j - k da base e desenrola o laço mais interno em 4 cópias, com laço de resto para correção em qualquer N .
- **Loop Tiling:** bloqueia o espaço de iteração em cubos de lado 32, mantendo a ordem interna i - j - k *idêntica* à base. O lado 32 foi escolhido a partir da geometria da L1d, conforme a Seção 2.4.

2.3 Os dois níveis do Fator 2

Na alocação **estática** as matrizes são arrays de tamanho fixo e contíguos, declarados em escopo de arquivo. O escopo de arquivo é obrigatório e não estilístico: 11,44 MiB excedem o limite de pilha de 8 MiB desta máquina, e em `-O0` o compilador não emite verificação de pilha, de modo que o estouro seria uma falha de segmentação sem diagnóstico.

Na alocação **dinâmica** as matrizes são `int**` com uma chamada de `malloc` por linha, portanto as linhas não são contíguas entre si.

Os quatro laços da família estática são *idênticos caractere por caractere* aos da família dinâmica. A única diferença entre as duas famílias é o tipo do parâmetro, `int m[N][N]` contra `int **m`, e portanto o endereçamento. Essa equivalência foi verificada por comparação automática do texto dos kernels.

A escolha de passar as matrizes como **parâmetro** em ambas as famílias, em vez de deixar o kernel estático ler diretamente variáveis globais, é o que mantém a comparação honesta. Em `-O0`, uma leitura de elemento custa três loads na forma `int **m`, dois na forma `int m[N][N]` e apenas um se o kernel lesse um global. A diferença real entre as duas alocações é um load de ponteiro dependente mais a perda de contiguidade entre linhas. A forma de parâmetro captura exatamente isso. A forma global adicionaria uma segunda unidade de diferença, que é artefato de passagem de parâmetro e não de alocação, e inflaria artificialmente o efeito do Fator 2 na métrica de loads.

2.4 Escolha do lado do bloco do Loop Tiling

A L1d desta máquina tem 48 KiB, linha de 64 bytes e 12 vias. O orçamento ingênuo, exigindo que três blocos de `int` caibam em 48 KiB, permitiria lado até 64. Esse cálculo está errado porque as linhas da matriz distam 4.000 bytes e nunca ficam alinhadas em 64 bytes, de modo que uma linha de bloco de T inteiros ocupa $\lceil (4T + 63)/64 \rceil$ linhas de cache. Com $T = 64$ o pior caso chega a 60 KiB e estoura a L1d. Com $T = 32$ o pior caso é 18,0 KiB, ou 37% da L1d, deixando folga para a pilha e para o carregamento do bloco seguinte.

O lado 32 também tem a vantagem de que $1.000 \bmod 32 = 8$, portanto o último bloco de cada dimensão tem largura 8 e as guardas de resto são executadas em toda execução. Um lado que dividisse N exatamente tornaria essas guardas código morto, nunca exercitado, embora continuassem necessárias para correção.

2.5 Protocolo de medição

Todos os oito binários são compilados com o mesmo comando, sem qualquer otimização: `gcc -O0 -std=c11 -Wall -Wextra -Werror`. A ausência de `-O` é exigida pelo enunciado e é estrutural: em `-O0` o compilador não vetoriza nem aplica transformação de laço, portanto as quatro variantes escritas à mão são genuinamente o que se mede. O Fator 1 é o código fonte, não o compilador.

O tempo de resposta é medido com `clock_gettime(CLOCK_MONOTONIC)` imediatamente antes e depois da chamada do kernel, e é registrado em nanossegundos inteiros. O preenchimento das matrizes de entrada e a zeragem de C acontecem *antes* do relógio, e os quatro kernels não zeram C . Isso mantém fora da região cronometrada tanto o custo do gerador pseudoaleatório quanto as faltas de página de primeiro toque, cujo custo difere entre as duas famílias de alocação e seria atribuído ao Fator 2 pelo motivo errado.

Os quatro contadores são coletados com `perf stat` envolvendo o processo. Foram adotadas as seguintes medidas de controle de variabilidade:

- execuções fixadas em um núcleo de desempenho com `taskset -c 0`
- uma execução de aquecimento por célula, descartada
- ordem de execução das 80 medições **randomizada** com semente fixa e persistida em arquivo, o que protege os efeitos dos fatores de se confundirem com deriva térmica progressiva
- 10 réplicas por célula, atendendo ao mínimo de dez do enunciado.

2.6 Nomenclatura dos eventos em CPU híbrida

A máquina de medição tem microarquitetura híbrida, com dois PMUs distintos, `cpu_core` para os núcleos de desempenho e `cpu_atom` para os de eficiência. A documentação do `perf-stat` adverte que um evento disponível nos dois PMUs faz o `perf` criar *dois* eventos automaticamente, um por PMU. Como as execuções são fixadas em um núcleo de desempenho, o evento gêmeo do `cpu_atom` nunca é escalonado e retorna `<not counted>`, valor que um analisador ingênuo leria como zero. Por isso todos os eventos foram qualificados explicitamente como `cpu_core/EVENTO/`.

Além disso, nesta máquina os aliases `L1-dcache-*` não aparecem em `perf list`, porque são eventos legados do tipo `PERF_TYPE_HW_CACHE` codificados pelo próprio analisador do `perf` e não pelas tabelas de eventos do sistema. Para não depender de suposição sobre o que esses aliases realmente contam, foi feita uma validação cruzada contra os eventos nativos correspondentes, medindo a razão entre as duas contagens sobre a mesma carga.

Tabela 2: Validação cruzada dos aliases genéricos contra os eventos nativos.

Alias usado	Evento nativo comparado	Razão
L1-dcache-loads	mem_inst_retired.all_loads	1,0000
L1-dcache-load-misses	mem_load_retired.l1_miss	1,3050
branch-instructions	br_inst_retired.all_branches	1,0000
branch-misses	br_misp_retired.all_branches	1,0335

A razão unitária confirma que `L1-dcache-loads` é exatamente `mem_inst_retired.all_loads` e que `branch-instructions` é exatamente `br_inst_retired.all_branches`. Já `L1-dcache-load-misses` conta 1,3050 vezes o valor de `mem_load_retired.l1_miss`, ou seja **não** é o mesmo evento: mede algo mais amplo que apenas misses de load retirados. Essa é uma limitação declarada da métrica, e não altera as comparações entre células, porque o mesmo evento foi usado em todas as 80 medições.

2.7 Verificação de validade da comparação

Comparar tempos entre oito variantes de código só faz sentido se as oito calcularem o mesmo resultado. Duas verificações independentes foram feitas.

A primeira é interna ao conjunto de dados: cada execução imprime a soma dos N^2 elementos de C , e as 80 medições reportam um único valor, 16.251.093.877.953. A segunda é mais forte e roda fora do caminho de medição: uma compilação paralela dos oito binários imprime a matriz C completa, e o resumo criptográfico das oito saídas é idêntico. Essa segunda verificação compara todos os 1.000.000 de elementos, e não apenas um resumo de 64 bits deles.

Também foi verificado que todos os elementos de C ficam dentro do intervalo analítico $[0, 65.025.000]$, limite superior dado por $N \cdot 255^2$, o que descarta transbordo de inteiro. Esse é um erro que os resumos concordantes *não* detectariam, porque corromperia os oito kernels de forma idêntica.

Por fim, o percentual de tempo com contador ativo foi 100% nas 80 medições, o que comprova que nenhum valor sofreu multiplexação. Um contador multiplexado é escalado pelo `perf`, e o número resultante é extrapolação e não medição.

3 Análise 1: comparação do tempo de resposta

A Tabela 3 traz a média e o intervalo de confiança a 95% do tempo de resposta nos oito experimentos. O intervalo é o intervalo t por célula, $\bar{x} \pm t_{0,975} \cdot s/\sqrt{10}$, com 10 réplicas, 9 graus de liberdade e $t_{0,975} = 2,2622$. Essa escolha atende ao enunciado, que pede intervalo para cada métrica em *cada* experimento. Um intervalo com erro quadrático médio agrupado seria mais estreito, porém válido apenas sob homocedasticidade, pressuposto que a Seção 5 rejeita.

Tabela 3: Tempo de resposta do kernel, em milissegundos. Média, semiamplitude do intervalo de confiança a 95%, limites do intervalo e coeficiente de variação.

Exp.	Técnica	Alocação	Média	$\pm$ IC	IC inf.	IC sup.	CV (%)
1	base	estatica	4.319,4	71,0	4.248,4	4.390,4	2,30
2	base	dinamica	5.469,0	200,1	5.268,8	5.669,1	5,12
3	interchange	estatica	3.180,5	35,0	3.145,5	3.215,5	1,54
4	interchange	dinamica	4.173,8	139,3	4.034,5	4.313,2	4,67
5	unrolling	estatica	4.509,8	130,9	4.378,9	4.640,8	4,06
6	unrolling	dinamica	5.364,8	167,0	5.197,8	5.531,7	4,35
7	tiling	estatica	4.052,4	69,1	3.983,3	4.121,6	2,39
8	tiling	dinamica	5.021,3	103,5	4.917,8	5.124,8	2,88

A Figura 1 apresenta a mesma informação graficamente, com as barras de erro do intervalo de confiança. Os experimentos aparecem na ordem do enunciado e não ordenados por valor, para que a estrutura do planejamento permaneça legível.

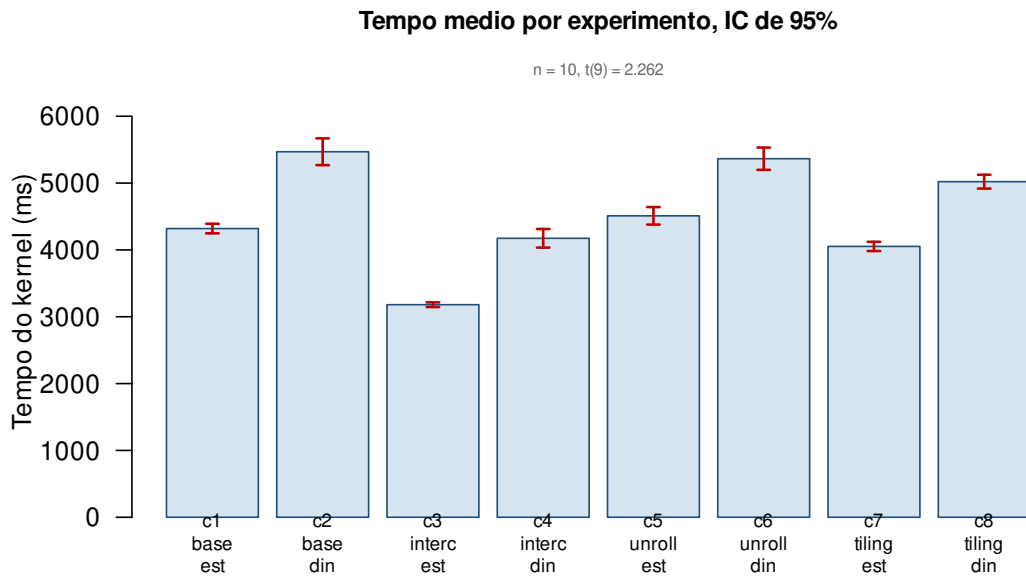


Figura 1: Tempo médio de resposta nos oito experimentos, com barras de erro do intervalo de confiança a 95%.

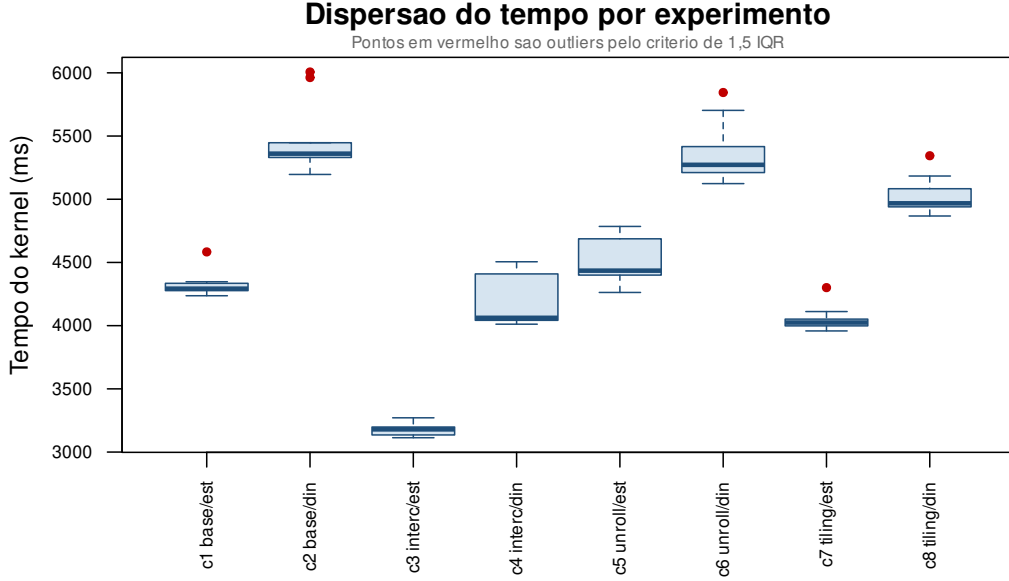


Figura 2: Dispersão do tempo de resposta por experimento. Os pontos em vermelho são valores atípicos pelo critério de 1,5 vez a amplitude interquartil.

3.1 Leitura dos resultados

O coeficiente de variação do tempo ficou entre 1,54% e 5,12%. As comparações par a par de Tukey, com correção para as 28 comparações possíveis, apontam 24 pares significativos a 5%, o que legitima a leitura direta das diferenças no gráfico de barras.

Três resultados se destacam.

Loop Interchange é a técnica mais eficaz, com ganho de $1,36\times$ na alocação estática e $1,31\times$ na dinâmica sobre o caso sem otimização. O motivo é direto: trocar j e k elimina o passeio por coluna de B , substituindo um acesso com passo de 4.000 bytes por um acesso sequencial.

Loop Tiling rende bem menos, $1,07\times$ e $1,09\times$. Isso é coerente com o fato de que o bloqueio foi aplicado sobre a ordem i - j - k da base, mantendo o passeio por coluna dentro de cada bloco. O bloqueio reduz o alcance desse passeio, mas não o elimina como o interchange faz.

Loop Unrolling não acelera, e na alocação estática chega a atrasar, $0,96\times$ contra a base. O resultado não é defeito de implementação: a contagem de instruções de branch caiu de $1,020\times 10^9$ para $0,272\times 10^9$, ou seja o desenrolamento de fator 4 funcionou exatamente como pretendido. O que ocorre é que, em -00, o custo dos recálculos de endereço para $k + 1$, $k + 2$ e $k + 3$ supera a economia de controle de laço. A transformação reduz desvios sem reduzir tempo.

Quanto ao Fator 2, a alocação dinâmica é em média $1,247\times$ mais lenta que a estática, e essa ordenação se mantém em todas as quatro técnicas.

3.2 Modelo fatorial completo, como suplemento

O enunciado pede apenas a comparação dos oito tempos, mas com os dados disponíveis é possível decompor a variação do tempo usando os quatro níveis de técnica ao mesmo tempo. A Tabela 4 traz esse modelo, ajustado sobre $\log_{10}$ do tempo porque a dispersão cresce com a média.

Tabela 4: Modelo fatorial completo 4×2 sobre $\log_{10}$ do tempo de resposta. Suplemento além das duas análises pedidas.

Fonte	gl	SS	%	MS	F	p
Técnica	3	0,21845	50,96	0,072817	305,64	$< 2 \times 10^{-16}$
Alocação	1	0,18834	43,94	0,188338	790,52	$< 2 \times 10^{-16}$
Técnica $\times$ Alocação	3	0,00469	1,09	0,001562	6,56	0,0006
Resíduos	72	0,01715	4,00	0,000238	—	—

A técnica responde por 50,96% da variação do tempo e a alocação por 43,94%. A interação entre os dois responde por apenas 1,09%, o que significa que, para o tempo, os dois fatores agem de forma quase aditiva: a escolha da melhor técnica é praticamente a mesma independentemente da alocação. O termo residual de 4,00% é a variabilidade de execução da máquina.

4 Análise 2: influência dos fatores nas variáveis de cache e de branch

Esta seção responde a segunda pergunta do enunciado. São dois planejamentos 2×2 independentes em resposta, cada um com 10 réplicas por célula, portanto 40 observações e 36 graus de liberdade de erro.

A codificação dos contrastes é $x_A = -1$ para alocação estática e $+1$ para dinâmica, e $x_B = -1$ para o caso sem otimização e $+1$ para a técnica sob teste. O modelo é

$$y = q_0 + q_A x_A + q_B x_B + q_{AB} x_A x_B + e, \quad (1)$$

e a variação atribuída a cada efeito é $SS_i = 2^2 \cdot r \cdot q_i^2$.

Os efeitos foram calculados por dois caminhos independentes, a matriz de sinais e a análise de variância, e verificou-se por código que as somas de quadrados coincidem. Uma terceira rota, pelas médias marginais, reproduz SS_A e SS_B . Três caminhos concordantes constituem prova, enquanto um caminho isolado seria apenas afirmação.

4.1 Média e intervalo de confiança dos contadores

O enunciado exige média e intervalo de confiança para todas as métricas coletadas. A Tabela 5 traz os quatro contadores nos oito experimentos.

Tabela 5: Contadores de hardware nos oito experimentos, média e semiamplitude do intervalo de confiança a 95%. Loads e instruções de branch em 10^9 , misses em 10^6 .

Exp.	Técnica	Aloc.	L1d loads (10^9)	L1d misses (10^6)	br. inst. (10^9)	br. misses (10^6)
1	base	estatica	$17,051 \pm 0,0002$	$649,96 \pm 1,03$	$1,019 \pm 0,0001$	$1,042 \pm 0,002$
2	base	dinamica	$21,058 \pm 0,0005$	$136,30 \pm 2,53$	$1,021 \pm 0,0003$	$1,051 \pm 0,003$
3	interchange	estatica	$17,049 \pm 0,0001$	$3,55 \pm 0,31$	$1,018 \pm 0,0000$	$1,037 \pm 0,001$
4	interchange	dinamica	$21,055 \pm 0,0003$	$3,54 \pm 0,14$	$1,019 \pm 0,0002$	$1,043 \pm 0,003$
5	unrolling	estatica	$15,803 \pm 0,0003$	$457,90 \pm 1,24$	$0,271 \pm 0,0002$	$1,046 \pm 0,003$
6	unrolling	dinamica	$19,810 \pm 0,0003$	$176,55 \pm 2,17$	$0,273 \pm 0,0002$	$1,049 \pm 0,002$
7	tiling	estatica	$18,246 \pm 0,0001$	$2,50 \pm 0,04$	$1,115 \pm 0,0001$	$32,080 \pm 0,003$
8	tiling	dinamica	$22,252 \pm 0,0002$	$2,51 \pm 0,05$	$1,116 \pm 0,0002$	$2,182 \pm 0,097$

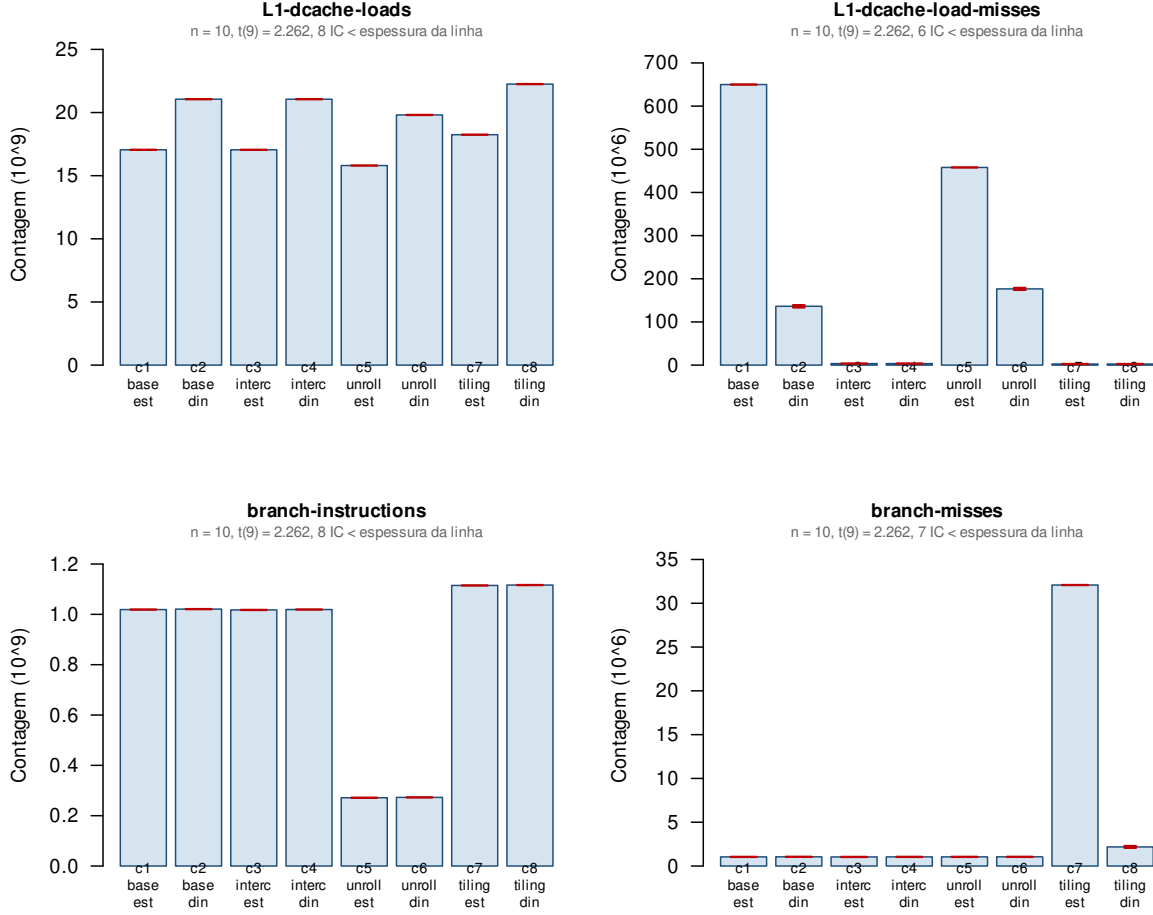


Figura 3: Os quatro contadores de hardware nos oito experimentos, com intervalo de confiança a 95%. Nos contadores de loads e de instruções de branch o intervalo é mais estreito que a espessura da linha, o que é informação e não defeito: essas duas métricas são função quase determinística do fluxo de instruções executado.

Uma observação metodológica importante decorre da Figura 3. O coeficiente de variação de **L1-dcache-loads** não passa de 0,0031% e o de **branch-instructions** não passa de 0,1133%. Essas duas métricas são essencialmente determinísticas. Em consequência, o erro quadrático médio das análises que as envolvem é minúsculo, os valores de F são enormes e praticamente todo valor de p resulta inferior a 2×10^{-16} . **A significância estatística passa a ser garantida e portanto não informativa.** O que interpreta este experimento é o tamanho do efeito e a alocação percentual de variação, não a rejeição de hipótese nula. Esse ponto reaparece de forma concreta na Seção 4.5.

4.2 Validação das contagens contra forma fechada

Antes de interpretar, vale confirmar que os contadores medem o que se supõe. A contagem de instruções de branch é previsível analiticamente: o kernel base executa cerca de $N^3 + N^2 + N \approx 1,001 \times 10^9$ desvios de condição de laço, e o desenrolamento de fator 4 reduz o laço interno para cerca de $0,251 \times 10^9$.

Os valores medidos são $1,020 \times 10^9$ para a base, $1,018 \times 10^9$ para o interchange, $0,272 \times 10^9$ para o unrolling e $1,116 \times 10^9$ para o tiling. Os três primeiros ficam a poucos por cento da previsão, e o excedente do tiling corresponde ao custo de controle dos laços de bloco. Essa concordância prova que o mapeamento de evento está correto e que os valores não estão escalados.

4.3 Análise de cache: experimentos 1, 2, 3 e 4

A variável de resposta é a **taxa de miss de L1d**, definida como `L1-dcache-load-misses` dividido por `L1-dcache-loads`. A taxa é preferível à contagem absoluta como resposta do planejamento porque é adimensional, mas a Seção 4.4 mostra que as duas leituras precisam ser apresentadas juntas.

Tabela 6: Alocação de variação e coeficientes do modelo, análise de cache sobre a taxa de miss de L1d, experimentos 1 a 4.

Fonte	SS	Influência (%)	q	p
A (alocação)	0,002510	25,3670	-0,007922	$< 2 \times 10^{-16}$
B (interchange)	0,004887	49,3919	-0,01105	$< 2 \times 10^{-16}$
AB (interação)	0,002497	25,2378	0,007901	$< 2 \times 10^{-16}$
Erro	$3,241 \times 10^{-7}$	0,0033	—	—

Tabela 7: Análise de variância, análise de cache.

Fonte	gl	SS	MS	F	p
A (alocação)	1	0,002510	0,002510	$2,788 \times 10^5$	$< 2 \times 10^{-16}$
B (interchange)	1	0,004887	0,004887	$5,429 \times 10^5$	$< 2 \times 10^{-16}$
AB (interação)	1	0,002497	0,002497	$2,774 \times 10^5$	$< 2 \times 10^{-16}$
Resíduos	36	$3,241 \times 10^{-7}$	$9,002 \times 10^{-9}$	—	—

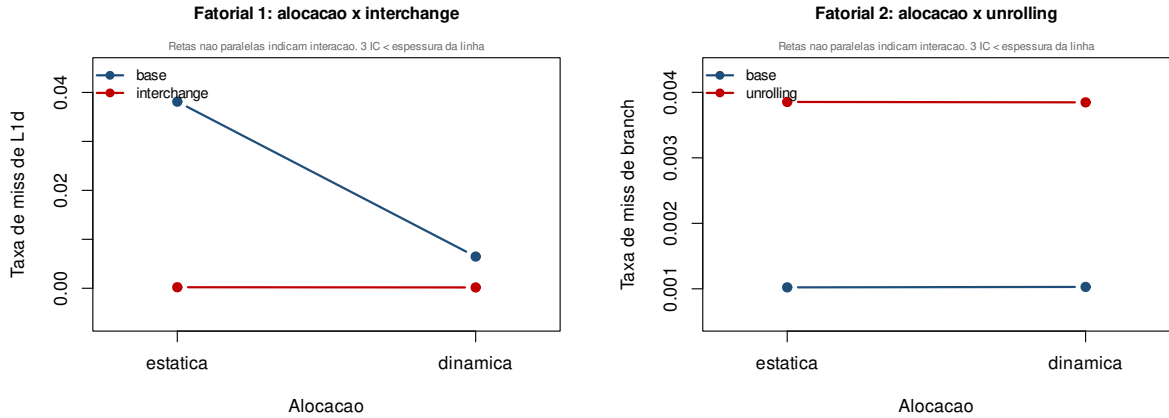


Figura 4: Gráficos de interação. À esquerda a análise de cache, à direita a de branch. Retas não paralelas indicam interação entre os fatores.

O resultado central é que os três termos têm influência da mesma ordem de grandeza: a técnica responde por 49,39%, a alocação por 25,37% e a **interação por 25,24%**. O erro fica em 0,0033%.

Uma interação que responde por um quarto da variação é o achado mais relevante desta análise, e o gráfico de interação da Figura 4 explica o mecanismo. No kernel sem otimização, a taxa de miss cai de 3,812% na alocação estática para 0,647% na dinâmica, uma diferença grande. No kernel com interchange, as duas alocações ficam praticamente empatadas, em torno de 0,019%.

Em outras palavras, **a alocação só importa enquanto existe o passeio por coluna**. O Loop Interchange elimina esse passeio, e ao fazê-lo torna a escolha da alocação irrelevante para a cache. Esse é o tipo exato de conclusão que um experimento de um fator por vez não produziria:

variando apenas a alocação com o kernel base, concluir-se-ia que a alocação tem forte influência sobre a cache, o que é falso para três das quatro técnicas.

4.4 Contagem absoluta contra taxa

A taxa de miss e a contagem absoluta de misses apontam em direções diferentes neste experimento, e apresentar apenas uma das duas seria enganoso.

O denominador da taxa depende da alocação. A família dinâmica executa, em média, $4,006 \times 10^9$ loads a mais que a estática, e essa diferença é notavelmente constante entre as quatro técnicas, variando apenas $0,0007 \times 10^9$ entre a maior e a menor. Essa constância indica que a diferença é estrutural, vinda da indireção de ponteiro em -00, e não do padrão de acesso à memória.

Quanto ao numerador, no kernel sem otimização a alocação estática registra $650,0 \times 10^6$ misses contra $136,3 \times 10^6$ da dinâmica. A alocação contígua **erra mais**, não menos, ao contrário da expectativa usual. O mesmo padrão aparece no unrolling, $457,9 \times 10^6$ contra $176,6 \times 10^6$. Já no interchange e no tiling, que não fazem passeio por coluna, as duas alocações ficam próximas, cerca de $3,6 \times 10^6$ e $2,5 \times 10^6$.

A hipótese mais plausível para esse comportamento é conflito de conjuntos na L1d. O passo entre linhas da matriz estática é 4.000 bytes, muito próximo do período de repetição de conjuntos da cache, que é 4.096 bytes. Um passeio por coluna de N elementos distribui-se então por apenas 64 conjuntos, cerca de 15,6 linhas por conjunto, acima das 12 vias disponíveis, o que força misses de conflito. Na alocação dinâmica cada linha vem de uma chamada de `malloc` independente, o que altera os deslocamentos e desmancha essa regularidade.

Esta é uma hipótese e não uma conclusão deste experimento. O planejamento executado não a testa. O teste direto seria acrescentar preenchimento ao passo entre linhas da matriz estática, deslocando-o do período de repetição de conjuntos, e verificar se a contagem de misses cai. Fica registrado como trabalho futuro.

4.5 Análise de branch: experimentos 1, 2, 5 e 6

A variável de resposta é a **taxa de miss de branch**, definida como **branch-misses** dividido por **branch-instructions**.

Tabela 8: Alocação de variação e coeficientes do modelo, análise de branch sobre a taxa de miss de branch, experimentos 1, 2, 5 e 6.

Fonte	SS	Influência (%)	q	p
A (alocação)	$2,378 \times 10^{-12}$	0,0000	$2,438 \times 10^{-7}$	0,8501
B (unrolling)	$7,980 \times 10^{-5}$	99,9965	0,001412	$< 2 \times 10^{-16}$
AB (interação)	$4,298 \times 10^{-10}$	0,0005	$-3,278 \times 10^{-6}$	0,0149
Erro	$2,364 \times 10^{-9}$	0,0030	—	—

Tabela 9: Análise de variância, análise de branch.

Fonte	gl	SS	MS	F	p
A (alocação)	1	$2,378 \times 10^{-12}$	$2,378 \times 10^{-12}$	0,03622	0,8501
B (unrolling)	1	$7,980 \times 10^{-5}$	$7,980 \times 10^{-5}$	$1,215 \times 10^6$	$< 2 \times 10^{-16}$
AB (interação)	1	$4,298 \times 10^{-10}$	$4,298 \times 10^{-10}$	6,546	0,0149
Resíduos	36	$2,364 \times 10^{-9}$	$6,566 \times 10^{-11}$	—	—

O resultado é o oposto do anterior em estrutura. A técnica responde por 99,9965% da variação, e **a alocação não tem efeito algum**: influência de 0,0000% e $p = 0,850$, o único efeito não significativo de todo este trabalho.

Esse resultado é fisicamente esperado e serve como validação do experimento. A predição de desvios depende da estrutura do laço, não de onde a memória foi alocada. As médias confirmam: a taxa de miss de branch é praticamente idêntica entre as duas alocações dentro de cada técnica, e o gráfico de interação da Figura 4 mostra duas retas horizontais e paralelas.

4.6 Um efeito significativo e irrelevante ao mesmo tempo

A interação desta análise ilustra de forma concreta o ponto metodológico levantado após a Figura 3. O termo AB é estatisticamente significativo, com $p = 0,0149$, e ao mesmo tempo responde por 0,0005% da variação.

Significância sem relevância prática. Com uma resposta quase determinística, qualquer diferença sistemática, ainda que da ordem de 10^{-6} , é detectada como significativa porque o erro é ainda menor. É por isso que este relatório apresenta a alocação percentual de variação como resultado principal e o valor de p como informação secundária.

Vale notar também que o desenrolamento *aumenta* a taxa de miss de branch, que passa de 0,1026% no caso sem otimização para 0,3850% com Loop Unrolling. Isso não indica piora da predição: o desenrolamento reduziu o *denominador* em quatro vezes, conforme já verificado contra a forma fechada. A contagem absoluta de branch misses permanece da mesma ordem, e a razão sobe porque há menos desvios totais.

5 Validação dos pressupostos

As duas análises de variância pressupõem normalidade, homocedasticidade e independência dos resíduos. A Tabela 10 apresenta os testes, e as Figuras 5 e 6 os diagnósticos gráficos correspondentes.

Tabela 10: Verificação dos pressupostos nas duas análises.

Teste ou estatística	Cache (1–4)	Branch (1, 2, 5, 6)
Shapiro-Wilk, p	0,001268	0,03839
Bartlett, p	$7,751 \times 10^{-12}$	$5,150 \times 10^{-6}$
Fligner-Killeen, p	$1,822 \times 10^{-4}$	$6,128 \times 10^{-4}$
Razão máxima entre variâncias	320,2	37,7
Autocorrelação de defasagem 1	−0,012	0,059
Durbin-Watson	1,995	1,876
Ljung-Box (5 defasagens), p	0,122	0,232
Maior $ r $ entre sinal e ordem	0,168	0,276

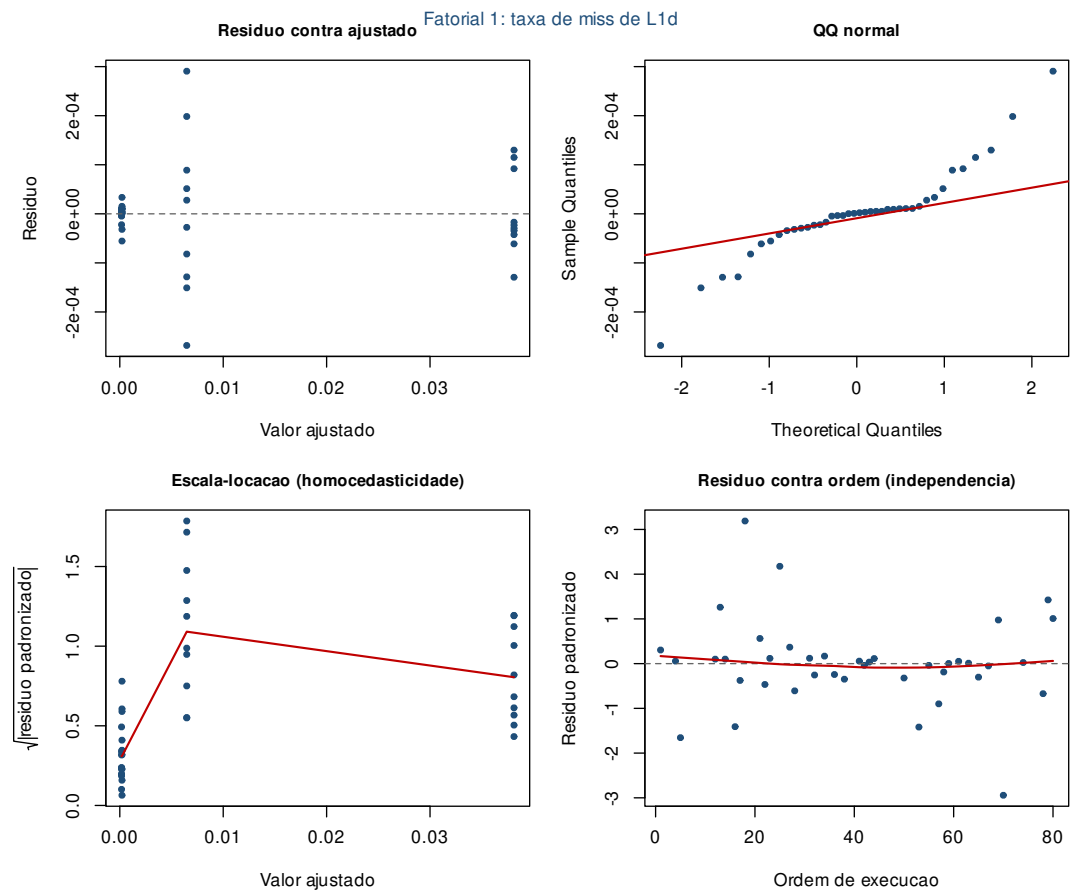


Figura 5: Diagnóstico de resíduos da análise de cache.

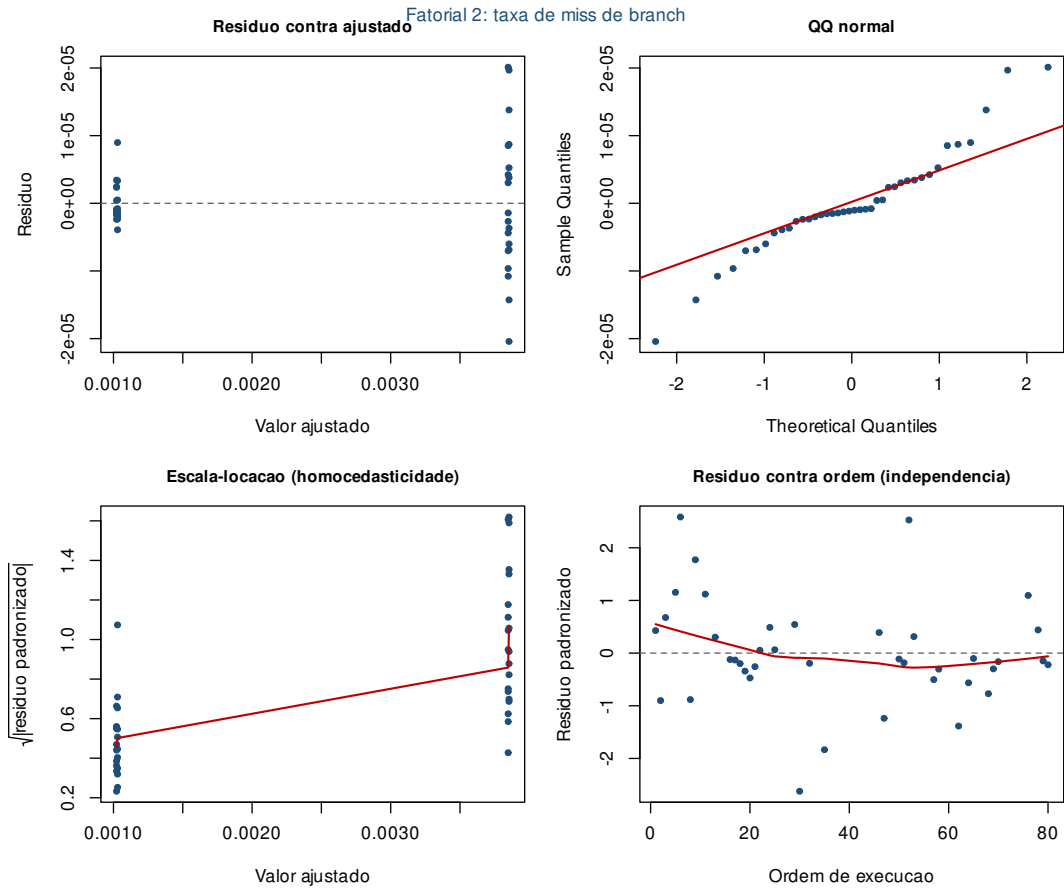


Figura 6: Diagnóstico de resíduos da análise de branch.

5.1 Independência: sustenta-se

A autocorrelação de defasagem 1 dos resíduos é -0,012 na análise de cache e 0,059 na de branch, e a estatística de Durbin-Watson fica em 1,995 e 1,876, ambas muito próximas do valor 2 esperado sob independência. O teste de Ljung-Box não rejeita em nenhuma das duas.

Além disso, nenhuma coluna de sinal está significativamente alinhada com a ordem de execução, sendo o maior módulo de correlação 0,168 e 0,276. Isso importa porque é o que decide se uma eventual deriva temporal enviesa as estimativas ou apenas infla o erro. Não havendo alinhamento, a deriva entra no termo de erro, o que torna os testes conservadores e não otimistas.

A randomização da ordem de execução é o que produz esse resultado.

5.2 Normalidade e homocedasticidade: rejeitadas

O teste de Shapiro-Wilk rejeita a normalidade dos resíduos nas duas análises, e os testes de Bartlett e de Fligner-Killeen rejeitam a igualdade de variâncias, com razão entre a maior e a menor variância de 320,18 e 37,73 vezes.

Ambas as rejeições eram esperadas, e por um motivo pouco usual. Como os contadores são quase determinísticos, os resíduos ficam concentrados em poucos valores distintos, quase discretos, e é essa forma que o Shapiro-Wilk sinaliza. A heterocedasticidade também tem origem física: a variância da célula sem otimização é ordens de grandeza maior que a da célula com interchange, porque o passeio por coluna é sensível a interferência do sistema enquanto o acesso sequencial não é.

Duas providências foram tomadas em vez de ignorar o problema.

Primeira, reanálise em escala logit. As duas respostas são proporções, portanto o estabilizador de variância adequado é $\log(p/(1 - p))$ e não o logaritmo simples. A Tabela 11 compara a alocação percentual nas duas escalas. O ponto essencial é que **a ordenação dos efeitos não muda** em nenhuma das duas análises, de modo que as conclusões da Seção 4 são robustas à escolha de escala. Na análise de branch a transformação ainda corrige a heterocedasticidade, elevando o p de Fligner-Killeen para 0,409.

Tabela 11: Influência percentual em escala bruta e em escala logit.

Análise	Fonte	% bruta	% logit
1	<i>A</i>	25,3670	4,7140
1	<i>B</i> (interchange)	49,3919	92,2390
1	<i>AB</i>	25,2378	2,9570
1	Erro	0,0033	0,0901
2	<i>A</i>	0,0000	0,0004
2	<i>B</i> (unrolling)	99,9965	99,9969
2	<i>AB</i>	0,0005	0,0010
2	Erro	0,0030	0,0017

Segunda, teste de permutação restrita de Freedman-Lane, com 10.000 permutações. O procedimento permuta os resíduos do modelo reduzido, isto é do modelo sem o efeito sob teste mas com todos os outros. Permutar a resposta bruta seria incorreto, porque a hipótese nula absorveria a variância dos demais efeitos. O teste dispensa completamente o pressuposto de normalidade.

Tabela 12: Valores de p da análise de variância e do teste de permutação de Freedman-Lane.

Análise	Efeito	p (ANOVA)	p (permutação)	Signif. por IC	Signif. por permutação
1	<i>A</i>	$< 2 \times 10^{-16}$	10.0×10^{-5}	sim	sim
1	<i>B</i>	$< 2 \times 10^{-16}$	10.0×10^{-5}	sim	sim
1	<i>AB</i>	$< 2 \times 10^{-16}$	10.0×10^{-5}	sim	sim
2	<i>A</i>	0,8501	0,8472	não	não
2	<i>B</i>	$< 2 \times 10^{-16}$	10.0×10^{-5}	sim	sim
2	<i>AB</i>	0,0149	0,0121	sim	sim

A concordância é total, 3 de 3 efeitos na análise de cache e 3 de 3 na de branch. Os valores de p da permutação rastreiam de perto os da análise de variância, inclusive no único efeito não significativo. O veredito de significância, portanto, não depende do pressuposto de normalidade que foi rejeitado.

6 Conclusão

Sobre a **Análise 1**, a comparação dos oito tempos de resposta mostra que Loop Interchange é a técnica mais eficaz, com ganho de até $1,36\times$, seguida de Loop Tiling com ganho modesto, enquanto Loop Unrolling não acelera e chega a atrasar em -00 , apesar de reduzir a contagem de desvios em quatro vezes conforme pretendido. A alocação dinâmica é $1,247\times$ mais lenta que a estática em todas as técnicas. No modelo suplementar, técnica e alocação respondem por 50,96% e 43,94% da variação do tempo, com interação de apenas 1,09%.

Sobre a **Análise 2**, as duas respostas se comportam de maneiras estruturalmente opostas, e isso é o resultado mais informativo do trabalho.

Na resposta de **cache**, os três termos são comparáveis, com técnica em 49,39%, alocação em 25,37% e interação em 25,24%. A interação significa que a influência da alocação sobre a cache existe apenas enquanto o kernel faz passeio por coluna. Loop Interchange remove esse passeio e com ele a relevância da alocação.

Na resposta de **branch**, a técnica responde por 99,9965% e a alocação por 0,0000%, sendo o único efeito não significativo do trabalho. Isso é fisicamente correto, pois a predição de desvios depende da estrutura do laço e não do esquema de alocação, e serve como verificação de sanidade do experimento.

A comparação entre as duas análises é o argumento prático a favor do planejamento fatorial completo. Variar um fator por vez, com o kernel base, levaria à conclusão de que a alocação influencia fortemente a cache. Essa conclusão é falsa para três das quatro técnicas, e só o termo de interação revela isso.

Se o objetivo for tempo de execução, a recomendação é Loop Interchange com alocação estática, o experimento 3, que foi o mais rápido dos oito.

7 Limitações

- **Uma única máquina, um único N e um único tamanho de bloco.** Os resultados de cache dependem da geometria específica da hierarquia de memória medida, e o valor de 32 foi escolhido para essa geometria.
- **As duas análises não são independentes entre si.** Ambas reutilizam os experimentos 1 e 2 como nível de referência, portanto compartilham 20 das 40 observações cada. Isso é inerente ao desenho pedido pelo enunciado, mas impede que os dois conjuntos de valores de p sejam citados como evidência independente.
- **O evento L1-dcache-load-misses não é exatamente miss de load retirado** nesta máquina, conforme a Tabela 2. Mede algo mais amplo, por fator 1,3050. Como o mesmo evento foi usado em todas as medições, as comparações entre células permanecem válidas, mas o valor absoluto não deve ser lido como taxa de miss de demanda.
- **Contadores cobrem o processo inteiro, o tempo cobre apenas o kernel.** O perf conta também a inicialização, o preenchimento das matrizes, o cálculo do resumo de verificação e, nos casos dinâmicos, mil chamadas de liberação de memória. Essas parcelas são pequenas frente ao kernel, mas diferem entre as duas famílias de alocação, portanto o desvio não é constante.
- **Multithreading simultâneo ativo.** O núcleo irmão do núcleo medido compartilha a L1d de 48 KiB e o preditor de desvios, e não há como desativá-lo sem privilégios administrativos. A máquina foi mantida ociosa durante a coleta.
- **Governador de frequência dinâmico.** A frequência varia entre 0,4 e 5,4 GHz sob demanda, o que é a origem principal da variabilidade do tempo de resposta. Os contadores de hardware não são afetados por isso, o que explica os coeficientes de variação muito baixos observados neles.
- **A explicação de conflito de conjuntos da Seção 4.4 é hipótese não testada**, e o teste correspondente está indicado como trabalho futuro.

8 Reprodutibilidade

Todo o material está no repositório, no diretório `profiling`. Os dados brutos das 80 medições estão em `dados/dados_brutos.csv`, com dicionário de colunas em `dados/dicionario.txt`. O ambiente completo de medição, incluindo modelo de processador, versões, estado do governador de frequência e resumos de verificação dos binários, está em `dados/ambiente.txt`. A sondagem e o mapeamento dos eventos do `perf` estão em `dados/eventos.txt`, e a ordem de execução sorteada em `dados/ordem_execucao.txt`. A saída do `perf` de cada uma das 80 execuções foi preservada em `dados/perf/` como trilha de auditoria.

A coleta é reproduzida com `bash experimento/run_experimento.sh` e a análise com `Rscript analise/analise.R`. Todos os valores numéricos citados neste texto são gerados pelo script de análise na forma de macros, e não digitados manualmente, de modo que texto e dados não podem divergir.

Para a correção ortográfica e a revisão de coesão deste texto foram utilizadas ferramentas de inteligência artificial. O experimento, as medições, a análise estatística e as conclusões são de autoria dos alunos.